第5回：検索してから回答する

キーワード検索でRAGの流れを学ぶ

今日のゴールと流れ

質問から検索語を取り出し、一致した数で資料を選ぶ仕組みを説明できる。
検索結果を見て、検索と生成のどちらに問題があるかを切り分けられる。

内容
RAGの流れ、準備と質問時の処理
検索語の取り出し（Janome）
一致した数で資料を並べる
穴埋め：点数のループ、並べ替えと上位件数
見つからない質問、提出

API利用時は毎回、起動するターミナルで：
`export OPENAI_API_KEY=sk-********`
終了時は停止して `unset OPENAI_API_KEY` 。

今日の進め方

段階	すること
1	`sample-app/keyword-app/` の完成版を動かす
2	コードの説明を聞く
3	`session05/exercise/` の穴あきコードを埋めて動かす
4	コミットしてプッシュする
5	GitHubのURLをフォームで提出する

穴あきコードの答えは `sample-app/keyword-app/` です。
アプリは前回と同じです。今回は `search.py` と `main.py` の `ask()` を読みます。

RAGとは

Retrieval-Augmented Generation：検索拡張生成。

質問に関係する情報を検索し、その情報を入力に加えて生成します。
この教材では、登録した授業資料の中を検索します。

RAGはインターネット検索だけを指す言葉ではありません。
今回は単語の一致で探します。次回はベクトルで探します。
どちらも、検索して生成する構成は同じです。

準備と質問時の処理

準備するとき（ init_db.py ）	質問を受けたとき（ main.py ）
スライドを1枚ずつに分ける	質問から検索語を取り出す
Janomeで単語に分ける	単語が一致する資料を探す
見出し・本文・単語をDBに保存する	上位の資料をLLMへ渡す
	回答と渡した資料を表示する

準備は資料を直したときに1回、検索と生成は質問ごとに行います。
資料の形態素解析は準備のときに済ませ、質問のときは質問だけを解析します。

ask() の3段階

質問：「学習と推論の違いは？」

段階	関数	結果
① 検索語を取り出す	`extract_words()`	`['学習', '推論', '違い']`
② 資料を探す	`search_documents()`	点数付きの資料、多い順に最大3件
③ LLMに答えさせる	`ask_with_documents()`	資料を根拠にした回答

①と②は `search.py` 、③は `llm.py` にあります。
`main.py` の `ask()` が、この順に呼びます。

完成版を動かす

```
source .venv/bin/activate
cd sample-app/keyword-app
python init_db.py
python main.py
```

ポート8001番をブラウザで開きます。

`init_db.py` は、36枚のスライドを単語に分けて `documents.db` に保存します。

「学習と推論の違いは？」と質問し、回答の下にある
「検索語」と「LLMに渡した資料」を見ます。

検索語を取り出す：extract_words()

```
def extract_words(text):
    words = []
    for token in tokenizer.tokenize(text):
        parts = token.part_of_speech.split(',')
        if parts[0] not in USE_PARTS:
            continue
        if parts[1] in SKIP_DETAILS:
            continue
```

Janomeの `tokenize()` が、文章を単語（トークン）に分けます。
各トークンには品詞が付いています。品詞で使う語を選びます。

品詞で選ぶ

定数	値	意味
`USE_PARTS`	名詞、動詞、形容詞	この品詞だけ使う
`SKIP_DETAILS`	非自立、代名詞、接尾	名詞でも除く細分類
`STOP_WORDS`	する、ある、いる、なる、できる、れる、られる	どの文章にも出る語

助詞（は・を・に）や記号（？）は `USE_PARTS` に無いので使いません。
「の」は非自立、「それ」は代名詞、「提出物」の「物」は接尾です。

基本形にそろえる

```
        word = token.base_form
        if word == '*':
            word = token.surface
        word = word.lower()

        if word in STOP_WORDS:
            continue
        if word not in words:  # 同じ語は1回だけ入れる
            words.append(word)
    return words
```

「閉じて」の「閉じ」は、基本形「閉じる」になります。
辞書に無い語は基本形が `'*'` なので、書かれた形（`surface`）を使います。
`lower()` で `SQLite` と `sqlite` を同じ語にします。

検索語の例

質問	取り出される検索語
学習と推論の違いは？	学習、推論、違い
TransformerのAttentionとは？	transformer、attention
SQLiteに保存するには？	sqlite、保存
この授業で使うモデルは何？	授業、使う、モデル
？？？	（無し）

検索語が1つも無いと、`main.py` は400エラー
「質問から検索語を取り出せませんでした」を返します。

資料側の単語

```
def add_words(documents):
    for document in documents:
        document['words'] = extract_words(
            document['title'] + '\n' + document['heading'] + '\n' + document['body'])
```

`init_db.py` が、質問と同じ `extract_words()` で資料の単語を作ります。
分け方が違うと、同じ言葉でも一致しなくなるためです。
単語のリストはJSONの文字列にして `words` 列に保存します。

題名「第1回：LLMイントロダクション」も含めるので、
`llm` は36件すべての資料に入っています。

検索のスコア

検索語が「学習、推論、違い」の場合：

資料の単語（抜粋）	一致した検索語の数
学習、推論、処理、データ、モデル	2
llm、言語、モデル、学習、文章	1
前、会話、覚える、名前、チャット	0

検索語1つにつき1点です。資料に何回出ても1点です。
0点は除き、点数の大きい順に並べます。

手計算する3つの資料

検索語は `学習` 、 `推論` 、 `違い` です。 `words` の値は実物の抜粋です。

資料	`words` の抜粋
12枚目「学習と推論」	学習、推論、処理、授業、データ、モデル、パラメータ、api
7枚目「LLMとは」	llm、規模、言語、モデル、大量、文章、学習、入力、予測
13枚目「LLMは前の会話を覚えていない」	前、会話、覚える、モデル、入力、出力、名前、佐藤

3つの資料の点数を予想してください。

手計算の解答

資料	学習	推論	違い	合計
12枚目	1	1	0	2
7枚目	1	0	0	1
13枚目	0	0	0	0

13枚目は0点なので候補から除きます。
「違い」は3つのどれにも無く、2枚目「今日のゴール」だけに含まれます。
この点数は、質問の答えが本文にある確率ではありません。

forとifで数える

```
results = []
for document in documents:
    score = 0
    for keyword in keywords:
        if keyword in document['words']:
            score += 1

    if score > 0:
        results.append({
            'id': document['id'],
            ...
            'score': score,
        })
```

`keywords` は質問の検索語、`document['words']` は資料の単語リストです。
`in` は、リストにその語があるかを調べます。

ループ中のscore

12枚目「学習と推論」を処理する例です。

タイミング	keyword	`words` に含まれる	score
開始	まだ取り出していない	未判定	0
1回目	学習	含まれる	1
2回目	推論	含まれる	2
3回目	違い	含まれない	2

次の資料を処理するときは、`score = 0` から始めます。
前の資料の点数を引き継ぐと、順位が変わってしまいます。

上位件数：top_k

```
    results.sort(key=get_score, reverse=True)
    return results[:top_k]


def get_score(result):
    return result['score']
```

`get_score()` は辞書から点数を取り出す関数です。 `sort()` に渡します。
`reverse=True` で大きい順に並べます。
`[:top_k]` は先頭から `top_k` 件まで取り出す書き方です。
`top_k` には `main.py` の `TOP_K = 3` が渡されます。

並べ替えと上位選択の結果

「学習と推論の違いは？」で一致する資料は6件です。
12枚目が2点、2・7・8・9・35枚目が1点です。

TOP_K	LLMに渡す資料	理由
1	12	先頭の1件
3	12、2、7	2点が1件、1点はID順に2件
5	12、2、7、8、9	1点はID順に4件

「TransformerのAttentionとは？」は9枚目（2点）と8枚目（1点）の2件だけです。
`TOP_K` が3でも2件しか渡りません。最大件数であって、必ず返る数ではありません。

同点の資料の扱い

`sort()` は、同じ点数なら元の順番を保ちます。
元の一覧は `load_documents()` がID順に読んだものです。
同点ならIDの小さい資料が先になります。

1点の資料が5件あっても、`TOP_K = 3` なら2件しか渡りません。
先に並んだことだけでは、答えに必要な資料とは判断できません。
画面の「一致した検索語の数」と本文を見て確かめます。

0件ならAPIを呼ばない

```
found = search_documents(keywords, documents, TOP_K)

if not found:
    result = {
        'answer': '関係する資料が見つかりませんでした。言葉を変えて質問してください。',
        'input_tokens': 0,
        'output_tokens': 0,
    }
else:
    result = ask_with_documents(question, found, MODEL)
```

1件も無いときは、資料を渡しても答えられないので送りません。
利用料金もかかりません。画面には「入力0トークン・出力0トークン」と出ます。

画面で見るもの

表示	中身
検索語	`extract_words()` が質問から取り出した単語
LLMに渡した資料	`search_documents()` が返した資料。`[資料ID]`、見出し、本文
一致した検索語の数	各資料の `score`
完了（入力…トークン）	APIの利用量。0件のときは0

回答に付いた `[資料12]` のようなIDと、渡した資料の本文を見比べます。
資料を渡しても、回答が正しいとは限りません。

実習1：検索語と一致数を観察

完成版 `sample-app/keyword-app/` で2つ質問します。

質問	検索語	渡る資料
学習と推論の違いは？	学習、推論、違い	12（2点）、2、7（1点）
TransformerのAttentionとは？	transformer、attention	9（2点）、8（1点）

- 回答の `[資料ID]` が、渡した資料のどれを指しているか
- 1点の資料は、回答の根拠になっているか

確認したら Ctrl+C で止め、`cd ../../session05/exercise` へ移動します。

実習2：点数のループを埋める

`session05/exercise/search.py` の `search_documents()` にある1つ目の `TODO`。

1. `python init_db.py` と `python main.py` を実行し、埋める前に質問する
2. どんな質問でも「関係する資料が見つかりませんでした」になることを見る
3. `for keyword in keywords:` から `score += 1` までの3行のコメントを外す
4. サーバーを再起動し、「学習と推論の違いは？」と質問する

埋める前は `score` が0のままなので、全資料が候補から外れます。
エラーにはならないので、画面の表示で気づく必要があります。

実習3：並べ替えと上位件数を埋める

同じ関数の2つ目の `TODO` 。実習2を埋めただけの状態で先に質問します。

埋める前の画面	原因
「LLMに渡した資料」が6件	`[:top_k]` が無く、全部返している
2、7、8、9、12、35 の順	`sort()` が無く、ID順のまま

1. `results.sort(key=get_score, reverse=True)` と `return results[:top_k]` のコメントを外す
2. 仮の `return results` の行を消す
3. 再起動して質問し、12、2、7 の3件が点数順に並ぶことを確かめる

実習4：TOP_K を変えて比較

`session05/exercise/main.py` の `TOP_K` を変え、再起動して同じ質問をします。

TOP_K	「学習と推論の違いは？」で渡る資料	見ること
1	12	回答は変わるか
3	12、2、7	1点の資料は使われたか
5	12、2、7、8、9	入力トークン数の増え方

一度に変える条件は `TOP_K` だけにします。
終わったら3に戻します。

実習5：見つからない質問

質問	検索語	起こること
校庭の桜はいつ咲く？	校庭、桜、咲く	0件。APIを呼ばず固定の文を返す
来年度の試験日は何月何日？	来年度、試験、月、何	32枚目が見つかるが、試験日は無い
LLMが前の話を忘れるのはなぜ？	llm、前、話、忘れる	「忘れる」は無いが「前」「話」で13枚目
LLMの記憶力は？	llm、記憶	`llm` で全36件が1点。1〜3枚目が渡る

0件のときは「入力0トークン・出力0トークン」を確かめます。
32枚目は、第1回の資料にこの質問が例として書かれているため見つかります。

0件と根拠不足の違い

検索結果	教材の処理	確認すること
0件	LLMを呼ばず固定の文を返す	利用量が0か
1件以上だが答えが無い	資料を渡して生成する	「資料からは確認できません」と答えるか
必要な根拠あり	資料を渡して生成する	根拠に沿って回答するか

「来年度の試験日は何月何日？」は2行目です。
資料が見つかったことと、答えが資料にあることは別です。

表記が違うと一致しない

13枚目には「覚えていない」と書かれています。「忘れる」とは一致しません。

質問	結果
LLMが前の話を忘れるのはなぜ？	他の語で13枚目が1位になる
LLMの記憶力は？	同点が36件。ID順で1〜3枚目が渡り、13枚目は渡らない

キーワード検索は、意味の近さを判定しません。
「保管」と「保存」のような言い換えも一致しません。
この失敗を、次回のベクトル検索と比較します。

発展：STOP_WORDS を増やす

「この授業で使うモデルは何？」の検索語は「授業、使う、モデル」です。
「使う」は多くの資料に出るので、除く例です。

1. `search.py` の `STOP_WORDS` に `'使う'` を足す
2. `python init_db.py` を実行し、資料側の単語も作り直す
3. `python main.py` で再起動し、検索語が「授業、モデル」になることを見る

資料側の単語も同じ `extract_words()` で作っているので、
`search.py` を変えたら `init_db.py` をやり直します。

検索と生成の失敗を分ける

検索結果	回答	最初に調べる場所
必要な根拠がない	誤答	検索語、`STOP_WORDS`、`TOP_K`
必要な根拠がある	誤答	指示文、渡した本文、モデルの出力
根拠がない	回答を控える	言い換えで見つかるか
根拠がある	正答	別の質問でも確かめる

検索結果を見ずに、指示文だけを変え続けないようにします。
画面の「検索語」と「LLMに渡した資料」が、切り分けの材料です。

失敗した場所の切り分け

観察した状態	まず確認すること
`janome` のimportで止まる	仮想環境と依存パッケージ
「documents.db がありません」	`python init_db.py` を実行したか
どの質問も「関係する資料が見つかりませんでした」	実習2のループを外したか
渡した資料が3件を超える、点数順でない	実習3の2行と仮の `return`
「質問から検索語を取り出せませんでした」	名詞を含む質問にする
「API呼び出しに失敗しました」	キー・モデル・利用枠
資料を直しても変わらない	`init_db.py` の再実行と再起動

確認問題

1. このアプリは、質問の文をそのまま資料の本文と比べている？
2. `TOP_K` を5にすると、必ず5件の資料がLLMに渡る？
3. 検索結果が0件のとき、APIは呼ばれる？
4. 「覚えていない」と書かれた資料は、「忘れる」という検索語で見つかる？

提出物と次回

`session05/exercise/` をコミットし、プッシュします。 `TOP_K` は3に戻しておきます。

```
cd ../..  # リポジトリのルートへ戻る
git add session05/exercise
git commit -m "第5回: キーワード検索"
git push
```

GitHubで `session05/exercise` を開き、そのURLをフォームから提出します。
`documents.db` はGitに入れません。

次回は、文章をベクトルにして意味の近さで検索します。